REPORT · 2026年10月18日

中小型文物三维建模自动化系统

全链路审计与跨层对齐

文保数字化研发组
2026年10月18日

目 录

一、审计方法与结论摘要 **3**

1.1 发现与修复一览 3

二、D1（致命）：项目名冒充文物编号 **4**

2.1 修复：反查而非假定 4

三、D2/D3：阶段终态断裂（灯常橙 + 进度少一格） **5**

3.1 单站 M2：返回 DONE，却不打日志 5

3.2 真跳过：只有 debug 级，界面看不见 5

四、D4：阶段清单漏了 P1 **6**

五、D5：版本号四处不一致 **7**

六、D6：小件上体素失效（算法与界面） **8**

七、D7/D8：隐蔽缺陷 **9**

八、验证与回归 **10**

8.1 端到端实跑（源码态） 10

8.2 日志回灌状态机 10

8.3 测试与静态检查 10

九、结论与下一步 **11**

一、审计方法与结论摘要

本次按**三层扫描法**遍历全管线，顺序是：**底层算法**（参数在真实尺度上是否仍成立）→ **中层工程路径**（产物目录 / manifest / 阶段注册表 / 版本号这些「跨模块契约」）→ **GUI 表现**（状态机能否观测到内核真实发生的每件事）。每一层都用可执行证据判定，不靠阅读印象。

序	手段	覆盖对象	产出
1	静态检查（pyflakes / AST 扫描）	src/h3d 全部 60 个模块 + tests	未定义名 2 处、静默吞异常 34 处、死变量 8 处
2	契约比对（代码级）	GUI 配置键与内核 dataclass、STAGE_ORDER 与 STAGE_REGISTRY、产物路径与 item_id、版本号四处来源	3 类跨层不一致
3	真实端到端运行（源码态，非 exe）	示例项目 16 视角 640 px，S1→M1→M2→M3→M4→M5→M6（+X1 跳过）	70~90 s 跑通，产出 obj/mtl/diffuse/glb + 报表 + manifest
4	日志回灌状态机	把真实 run.log 逐行喂给 logparse，统计每盏灯的终态	定位「灯常橙 / 进度少一格」的具体阶段

一句话结论：算法层没有算错，错在**三层之间的契约**。最致命的一条是GUI 把「项目名」当「文物编号」去拼产物路径——示例项目里两者同名（``demo_relic``）所以自测一直通过，而真实项目（项目 ``yyyyyyy`` / 文物 ``relic_001BMYyyyy``）必然不同名，于是**模型明明交付了、界面却说没有**。这与用户「跑了很久却拿不到一次完整输出」的体感完全吻合。

1.1 发现与修复一览

编号	层	缺陷	症状	级别
D1	中层与 GUI	用项目名当文物编号拼路径	预览空白 / 导出报无产物	致命
D2	内核与 GUI	单站 M2 返 DONE 却不打完成日志	M2 灯常橙，进度 7/10	高
D3	内核与 GUI	真跳过只 log.debug	跳过时该灯橙色卡死	高
D4	GUI 与内核	STAGE_ORDER 漏 P1	无灯 + 槽内 KeyError 被吞	高
D5	全局	版本号四处不一致（含模板写死）	报表 0.1.0 vs 安装包 1.1.0	中
D6	算法与 GUI	小件上体素 < 点间距致降采样失效	M1 前后点数几乎不变	中
D7	工程卫生	history.__all__ / Optional / 静默回写	星号导入报错、阈值无痕迹	低

二、D1（致命）：项目名冒充文物编号

管线的产物目录一律按 **item_id** 组织——即 ``00_raw`` 下的子目录名。而 GUI 的预览、贴图、导出、热力图四处代码都写成了 ``self.cfg.name``（**项目名**）：

```
# 修复前（gui/app.py）
root, name = self.cfg.root_dir, self.cfg.name      # name 是项目名
return [
    os.path.join(root, '80_export', name, f'{name}.obj'),
    ...
]
r = export_delivery_package(self.cfg.root_dir, self.cfg.name, out)

# 实际产物位置
#   80_export/relic_001BMYyyyy/relic_001BMYyyyy.obj
#   50_tex/relic_001BMYyyyy/relic_001BMYyyyy_diffuse.png
```

示例项目 ``create_demo_project`` 里 **项目名被设成与文物编号相同**（都是 ``demo_relic``），所以每次自测都通过；真实项目的项目名是``yyyyyyy``、文物是 ``relic_001BMYyyyy``，于是 ``80_export/yyyyyyy/`` 永远不存在——预览面板显示「未生成可预览的三维模型」，点导出弹「未找到交付产物（80_export）」，而交付物就在旁边的目录里。

2.1 修复：反查而非假定

新增 ``h3d.gui.items``，按可信度反查编号：**run_manifest 的 stages 键前缀**（本次真实处理过的）→ ``00_raw`` 子目录（内含文件者）→ 平铺文件名 → 空。``primary_item`` 在多件中优先选**已交付**的那件（``80_export/<item>`` 存在）。

GUI 位置	修复前	修复后
预览候选路径	80_export/项目名/项目名.obj	80_export/文物编号/文物编号.obj
贴图面板	50_tex/项目名	50_tex/文物编号
打开 3D	80_export/项目名 + 50_tex/项目名	按文物编号
导出交付包	export(root, 项目名, out) → 空包	export(root, [全部文物编号], out)
X1 热力图	load_x1_changes(root, 项目名)	load_x1_changes(root, 文物编号)

导出同时支持多件：单件保持 ``交付模型/<file>`` 的历史布局（老调用方无感），多件改为 ``交付模型/<item>/<file>``——否则几件都叫 ``model.obj`` 时会互相覆盖。另加一道保险：**只有报表、没有 80_export** 时仍返回空包，避免用户拿到一个「只有 report.html」的包却以为模型已导出。

三、D2/D3：阶段终态断裂（灯常橙 + 进度少一格）

界面唯一的输入是日志行。内核必须把每个阶段的**终态**写成一行可被解析的日志，否则那盏灯就停在「运行中」。审计发现两处断裂，且都发生在真实数据的常见路径上。

3.1 单站 M2：返回 DONE，却不打日志

```
# 修复前（runner/__init__.py::_stage_register）
if len(cleaned) < 2:
    return StageResult('M2', TaskStatus.DONE, [], {...})  # 直接 return
# 后面的 log.info('M2 完成：...') 永远走不到

# 界面侧后果
#  ◆ M2 开始：demo_relic  -> 灯变橙
#  （无任何终态行）        -> 灯不再变化
#  进度 = 已完成阶段数/10  -> 最多 7/10
```

中小型文物常常只有**一站**扫描（一次摆位拍完），这不是边缘情况，而是新场景的默认形态。修复：单站分支补一行 ``M2 完成：<item>（单站，无需配准，跳过 M2 直接进入 M3）``。

3.2 真跳过：只有 debug 级，界面看不见

可选阶段打了「◆ 开始」却因条件不满足返回 SKIPPED 时，编排层原来只写 ``log.debug``（INFO 级日志里看不到）。于是 S1 无照片、X1 无参考期、X2 无纹理图、P1 与 M4 重复——这些**正常且常见**的情形都会留下一盏橙色灯。

修复分三处联动：① 各 SKIPPED 返回点补 ``skip_reason``；② 编排层对**已宣布开始**的阶段打 ``○ X 跳过：<item>（原因）``；③ 界面新增 ``stage_skip_of`` 解析、灰灯 ``#7a7f87``，并把跳过计入进度。

环节	修复前	修复后
内核日志	debug（不可见）	INFO ○ X1 跳过：item（缺参考期 …）
GUI 解析	无此概念	stage_skip_of() → 'skipped'
阶段灯	橙色常亮	灰色（已终止）
进度条	该阶段不计入	计入（与完成等价）
报表	跳过原因无处可查	manifest 记 skipped + reason

四、D4：阶段清单漏了 P1

内核 ``STAGE_REGISTRY`` 有 10 个阶段（M1~M6 + S1/P1/X1/X2），界面 ``STAGE_ORDER`` 只有 9 个——缺 **P1**（photo_texture）。P1 一旦被配置启用：「◆ P1 开始」正则不匹配（无灯）、``self.lamps['P1']`` 抛 KeyError、而 KeyError 发生在 Qt 槽里只会打到 stderr（打包版 console=False 看不见），Qt 继续跑——**界面停在上一阶段，与卡死无法区分。**

修复：``STAGE_ORDER`` 补 P1（放在 M4 之后，它消费 S1 位姿、与 M4 的 photo 模式互斥）、四条正则同步加 ``P1``、``_on_stage`` 改为 ``self.lamps.get(sid)`` 并对未登记阶段写一行可见日志——**槽函数不允许抛异常。**

五、D5：版本号四处不一致

位置	修复前	说明
src/h3d/__init__.py	0.1.0	``h3d --version`` 输出
pyproject.toml	1.0.0	安装元数据
release/H3D_v1.1.0	1.1.0	实际发行版本
report/html_export.py	模板里写死「0.1.0」	**即使改了 __version__ 也不会变**

修复：``__version__`` 定为 1.1.0 并注明是唯一来源；pyproject 对齐；报表模板改为 ``{{ version }}``，渲染时传 ``__version__``。此后改版本只需动 ``__init__.py`` + ``pyproject.toml``。

六、D6：小件上体素失效（算法与界面）

尺度自适应按「包围盒对角线 / 基准 3.7 m」线性缩放距离类参数，**比例**是保住的；但真实扫描的**点间距不随器物尺寸同比缩小**。实测三档（同一点数密度，40 k 点）：

器物	对角线	因子	点间距	生效体素	体素/点间距
10 cm 级（铜镜/印章）	0.131 m	0.0354	0.96 mm	0.18 mm	0.19
25 cm 级（小件陶瓷）	0.328 m	0.0886	2.39 mm	0.44 mm	0.19
米级（基准）	3.694 m	0.9985	27.1 mm	4.99 mm	0.18

体素小于点间距时，**每个体素本来就最多一个点**——降采样是空操作。用户看到的是「M1 前后点数几乎一样」，无法判断是参数无效还是数据本来就好。既有保护（``apply_spacing_floor``）只覆盖 FPFH/RANSAC/ICP 三个半径，不覆盖体素，因为「体素小于点距」并不一定会失败（只是不省事）。

> **处理方式（不擅改取值）**：内核在「体素 < 点间距」时打 WARNING 并给出建议值（≥2× 点间距）；界面把档位语义改成「**基准器物上的米制值**」并加提示。刻意**不**强行把体素抬到点间距——那会悄悄吃掉用户想要的细节，属于「用一个错误掩盖另一个错误」。

七、D7/D8：隐蔽缺陷

位置	问题	后果
gui/history.py	__all__ 写 HISTORY_PATH，实际叫 _HISTORY_PATH	``from h3d.gui.history import *`` 直接 AttributeError
novel_calib.py	函数注解用 Optional 但未 import	``from __future__ import annotations`` 掩盖了它；一旦 get_type_hints 就炸
runner/__init__.py	novel 阈值回写 ``except: pass``	写盘失败时无 metrics、无 WARNING，下次仍从旧值起步
ctrl_auto / incremental_depth / geo_align	8 处死变量（img_by_name / photos / fx / K_inv / mask / verts …）	不影响结果，但说明有分支被重构后未清理

第三项按项目既有约定改为「写 metrics + WARNING」：``novel_threshold_error`` 落指标、日志点名原因。这是同类问题第三次出现（np 漏导入、P3 静默失效、本次），共识是**凡是 except 兜底赋值，必须同时留下可检索的痕迹。**

八、验证与回归

8.1 端到端实跑（源码态）

阶段	结果	备注
S1	16/16 张注册，1 相机，50.1 s	2 个子重建（[16, 2]），择优取 16 张那个
M1	1 站，8000 → 4363 点	voxel 0.0501、法线 k=20
M2	完成（单站，无需配准）	**本次修复点**：修复前无此行
M3	10039 顶点 / 19829 面	poisson depth=7
M4	photo 投影纹理，2 文件	UV 展开 10039 → 14222 顶点
M5	coverage 100%、rmse 27.4 mm、pass	参考过疏，中误差仅参考不判门
M6	obj + mtl + diffuse.png + glb	交付清单 1 件 1 已发布
X1	○ 跳过（缺参考期）	**本次修复点**：修复前静默

8.2 日志回灌状态机

把真实 run.log 逐行喂给 logparse，统计每个阶段灯的终态——这是「界面是否忠实反映内核」的直接判据：

阶段	修复前终态	修复后终态
S1 / M1 / M3 / M4 / M5 / M6	done	done
M2	**running（永不熄灭）**	done
X1	**running（永不熄灭）**	skipped
P1 / X2	无灯（未宣布）	无灯（未宣布，正确）
卡在运行中	2 盏	**0 盏**

8.3 测试与静态检查

项	结果
全量测试	622 项通过（新增 20 项：编号反查 11、阶段对齐 6、阶段日志与尺度告警 3）
pyflakes 未定义名	0（原 2：history.__all__、novel_calib Optional）
交付三件套	obj 3.1 MB / mtl 177 B / diffuse.png 1.0 MB / glb 1.5 MB
报表	HTML 7 阶段行 + 覆盖率/中误差表 + 版本号已对齐

九、结论与下一步

本次没有改动任何算法参数，也没有改管线编排逻辑；所有改动都落在**契约**上：路径用谁的编号、阶段终态怎么写、清单以谁为准、版本以谁为准。这类缺陷不会让程序崩溃，只会让用户**看不到已经存在的成果**——正是「跑很久却拿不到一次完整输出」的主因。

下一步	理由	优先级
重打包 exe + 安装包 + zip	本次 9 个源文件 + 新增 2 个模块（gui/items、gui/resources）尚未进包；r6.2 的 exe 仍带 D1~D5 全部缺陷	高
用真实小件（5~30 cm）跑一遍	验证 D6 告警是否在实机上出现，并据实际点间距定档位	高
M4 预览加 UV 纹理	matplotlib 预览只画三角面（纯色），交付物有贴图但预览看不出——属表现力问题，不影响交付	中
清理死变量 / 死分支	ctrl_auto、incremental_depth、geo_align 共 8 处	低
重复照片指纹检测	一个数据根下多件同名照片 = 隐藏的 N 倍耗时（历史教训）	中